

微服务篇

1. Spring Cloud 5大组件有哪些？

候选人：

在早期，Spring Cloud的五大组件通常指的是：

- **Eureka**：服务注册中心。
- **Ribbon**：客户端负载均衡器。
- **Feign**：声明式的服务调用。
- **Hystrix**：服务熔断器。
- **Zuul/Gateway**：API网关。

随着Spring Cloud Alibaba的兴起，我们项目中也融入了一些阿里巴巴的技术组件：

- 服务注册与配置中心：Nacos。
- 负载均衡：Ribbon。
- 服务调用：Feign。
- 服务保护：Sentinel。
- API网关：Gateway。

2. 服务注册和发现是什么意思？Spring Cloud 如何实现服务注册发现？

候选人：

服务注册与发现主要包含三个核心功能：服务注册、服务发现和服务状态监控。

我们项目中采用了Eureka作为服务注册中心，它是Spring Cloud体系中的一个关键组件。

- **服务注册**：服务提供者将自己的信息（如服务名称、IP、端口等）注册到Eureka。
- **服务发现**：消费者从Eureka获取服务列表信息，并利用负载均衡算法选择一个服务进行调用。
- **服务监控**：服务提供者定期向Eureka发送心跳以报告健康状态；如果Eureka在一定时间内未接收到心跳，将服务实例从注册中心剔除。

3. 我看你之前也用过nacos，你能说下nacos与eureka的区别？

候选人：

在使用Nacos作为注册中心的项目中，我注意到Nacos与Eureka的共同点和区别：

- **共同点：**两者都支持服务注册与发现，以及心跳检测作为健康检查机制。
- **区别：**
 - a. Nacos支持服务端主动检测服务提供者状态，而Eureka依赖客户端心跳。
 - b. Nacos区分临时实例和非临时实例，采用不同的健康检查策略。
 - c. Nacos支持服务列表变更的消息推送，使服务更新更及时。
 - d. Nacos集群默认采用AP模式，但在存在非临时实例时，会采用CP模式；而Eureka始终采用AP模式。

4. 你们项目负载均衡如何实现的？

候选人：

在服务调用过程中，我们使用Spring Cloud的Ribbon组件来实现客户端负载均衡。Feign客户端在底层已经集成了Ribbon，使得使用非常简便。

当发起远程调用时，Ribbon首先从注册中心获取服务地址列表，然后根据预设的路由策略选择一个服务实例进行调用，常用的策略是轮询。

5. Ribbon负载均衡策略有哪些？

候选人：

Ribbon提供了多种负载均衡策略，包括：

- **RoundRobinRule**：简单的轮询策略。
- **WeightedResponseTimeRule**：根据响应时间加权选择服务器。
- **RandomRule**：随机选择服务器。
- **ZoneAvoidanceRule**：区域感知的负载均衡，优先选择同一区域中可用的服务器。

6. 如果想自定义负载均衡策略如何实现？

候选人：

自定义Ribbon负载均衡策略有两种方式：

1. 创建一个类实现IRule接口，这将定义全局的负载均衡策略。
2. 在客户端配置文件中指定特定服务调用的负载均衡策略，这将仅对该服务生效。

7. 什么是服务雪崩，怎么解决这个问题？

候选人：

服务雪崩是指一个服务的失败导致整个链路的服务相继失败。我们通常通过服务降级和服务熔断来解决这个问题：

- **服务降级**：在请求量突增时，主动降低服务的级别，确保核心服务可用。
- **服务熔断**：当服务调用失败率达到一定阈值时，熔断机制会启动，防止系统过载。

8. 你们的微服务是怎么监控的？

候选人：

我们项目中采用了**SkyWalking**进行微服务监控：

1. SkyWalking能够监控接口、服务和物理实例的状态，帮助我们识别和优化慢服务。
2. 我们还设置了告警规则，一旦检测到异常，系统会通过短信或邮件通知相关负责人。

9. 你们项目中有没有做过限流？怎么做的？

候选人：

在我们的项目中，由于面临可能的突发流量，我们采用了限流策略：

- **版本1**：使用**Nginx**进行限流，通过漏桶算法控制请求处理速率，按照IP进行限流。
- **版本2**：使用**Spring Cloud Gateway**的**RequestRateLimiter**过滤器进行限流，采用令牌桶算法，可以基于IP或路径进行限流。

10. 限流常见的算法有哪些？

候选人：

常见的限流算法包括：

- **漏桶算法**：以固定速率处理请求，平滑突发流量。
- **令牌桶算法**：按照一定速率生成令牌，请求在获得令牌后才被处理，适用于请求量有波动的场景。

11. 什么是CAP理论？

候选人：

CAP理论是分布式系统设计的基础理论，包含一致性(Consistency)、可用性(Availability)和分区容错性(Partition tolerance)。在网络分区发生时，系统只能在一致性和可用性之间选择其一。

12. 为什么分布式系统中无法同时保证一致性和可用性？

候选人：

在分布式系统中，为了保证分区容错性，我们通常需要在一致性和可用性之间做出选择。如果系统优先保证一致性，可能需要牺牲可用性，反之亦然。

13. 什么是BASE理论？

候选人：

BASE理论是分布式系统设计中CAP理论中AP方案的延伸，强调通过基本可用、软状态和最终一致性来实现系统设计。

14. 你们采用哪种分布式事务解决方案？

候选人：

我们项目中使用了**Seata**的AT模式来解决分布式事务问题。AT模式通过记录业务数据的变更日志来保证事务的最终一致性。

15. 分布式服务的接口幂等性如何设计？

候选人：

我们通过Token和**Redis**来实现接口幂等性。用户操作时，系统生成一个Token并存储在Redis中，当用户提交操作时，系统会验证Token的存在性，并在验证通过后删除Token，确保每个Token只被处理一次。

16. xxl-job路由策略有哪些？

候选人：

xxl-job支持多种路由策略，包括轮询、故障转移和分片广播等。

17. xxl-job任务执行失败怎么解决？

候选人：

面对任务执行失败，我们可以：

1. 选择故障转移路由策略，优先使用健康的实例执行任务。
2. 设置任务重试次数。
3. 通过日志记录和邮件告警通知相关负责人。

18. 如果有大数据量的任务同时都需要执行，怎么解决？

候选人：

我们可以通过部署多个实例并使用分片广播路由策略来分散任务负载。在任务执行代码中，根据分片信息和总数对任务进行分配。